Department of Electrical Engineering

ASIC/FPGA Chip Design

Laboratory Manual

ASIC/FPGA Chip Design Laboratory Manual

Dr. Mahdi Shabany

Spring 2025

Contents

Lab 7 - Image Processing on Zynq

Objective

This lab teaches students to implement image processing algorithms on an FPGA using the Zynq-7000 SoC, focusing on practical skills in the development of FPGAs and real-time image processing. Here is what students will learn:

- Interface with camera and display peripherals using the Zynq's programmable logic and processing system
- Design and verify custom Verilog modules for image processing tasks, specifically grayscale conversion and Sobel edge detection
- Integrate hardware accelerators into a larger system design, leveraging the AXI interconnect for data transfer
- Utilize simulation tools to verify the functionality of hardware modules before deployment
- Optionally, extend the system functionality to include features like saving processed images to external storage

Introduction

FPGAs are increasingly vital for real-time image processing due to their flexibility and efficiency, and this lab utilizes the Zynq-7000 SoC to implement Sobel edge detection. Students will capture and display live video, design and simulate Verilog modules for grayscale conversion and edge detection, and integrate them into a hardware design for real-time processing. An optional task involves saving processed images to an SD card, offering practical experience in FPGA development for applications like autonomous vehicles and medical imaging.

Instructions

1.1 System Setup and Bootloader Creation

Start your image processing project in part 1.1 by setting up the Zynq-7000 SoC platform. You will work with a template project to capture live video, display it on an HDMI monitor, and save snapshots to an SD card. By the end, you will create a bootloader to auto-launch the camera project from the SD card, building key configuration skills for the weeks ahead.

1.1.1 Template Project Exploration

First, open the *the template project*. In this project, image data is captured from the camera board and the shot of each moment is placed in DDR3 memory using VDMA and then live displayed on the HDMI output. By pressing PS KEY1, the image of that moment (stored in the DDR3 frame buffer) is saved in BMP format on the micro SD. You can see the diagram of this project in the image below:

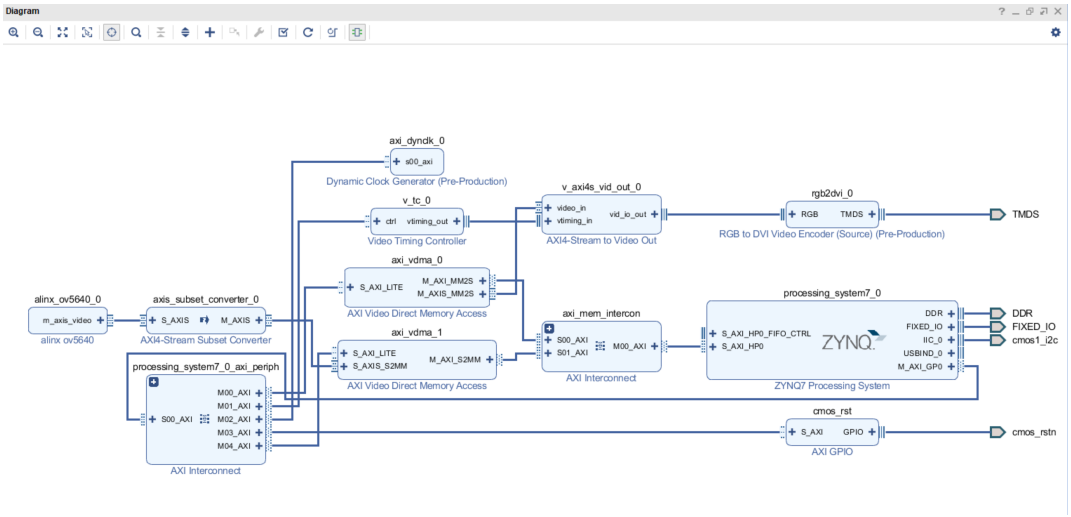


Figure 1.1: Block design of template project

(It is recommended to watch *video number 15* in advance to become familiar with AXI DMA, and *videos 18.b* and *18.c* to master the image transmission protocols in the FPGA and HDMI interface.) You can access the datasheet of the camera module (AN5642) through *this link*.

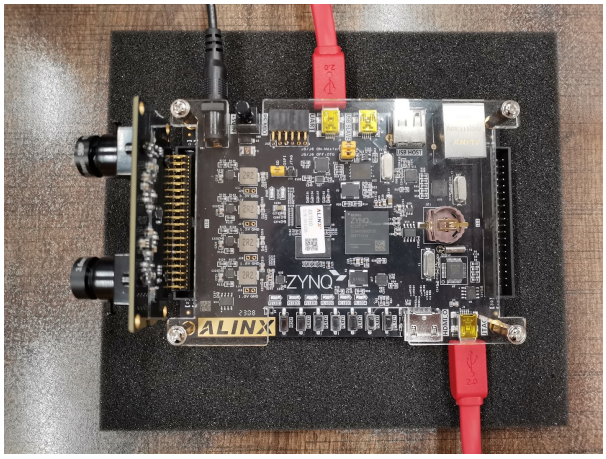


Figure 1.2: Top view of board and camera interface

After properly connecting the camera module to the board (as depicted in the above images) and connecting the HDMI output to the monitor, you can proceed to execute the project according to the following steps:

- Insert the SD card into the SD card holder on the back of the FPGA development board, turn on the power, and turn on the "putty".
- Download the program, after the image is displayed, press the button (PS KEY1), the PS LED1 will light when writing SD, and will be extinguished after writing.
- In putty, you can see the print information (After pressing PS KEY1 and taking a photo):

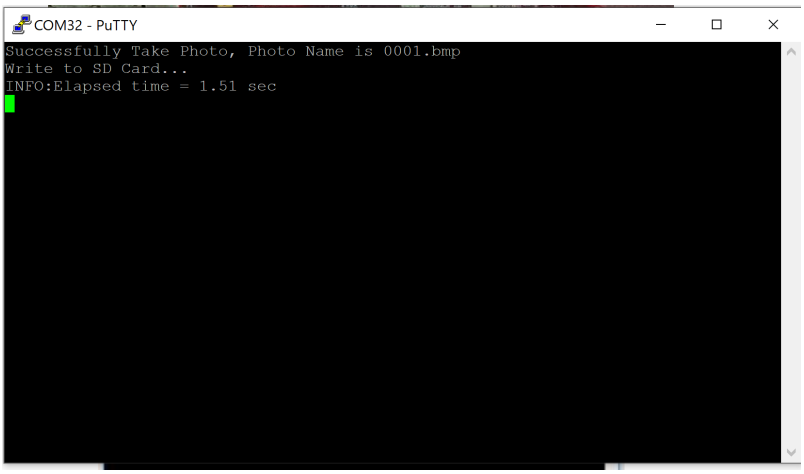


Figure 1.3: Output of Putty

- Remove the SD card after powering off, and you can see the BMP picture captured on the SD card in the computer:

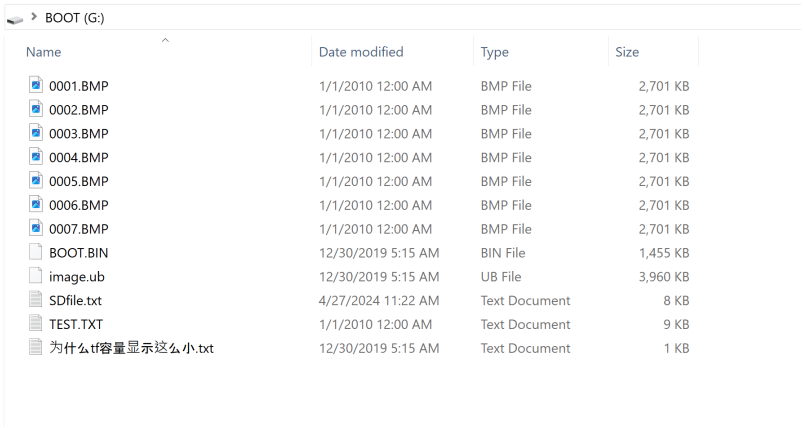


Figure 1.4: File manager

- Now open and show your picture and its dimensions

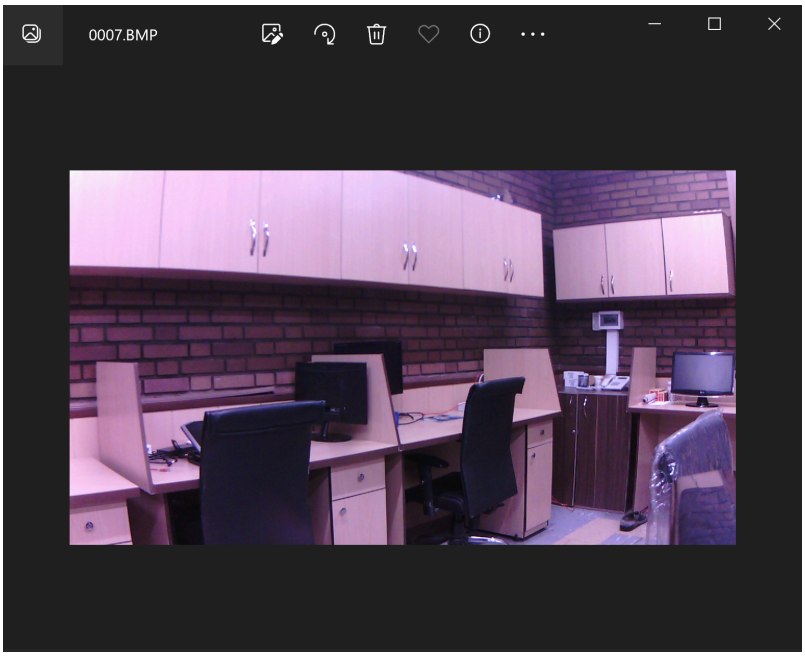


Figure 1.5: Example picture result

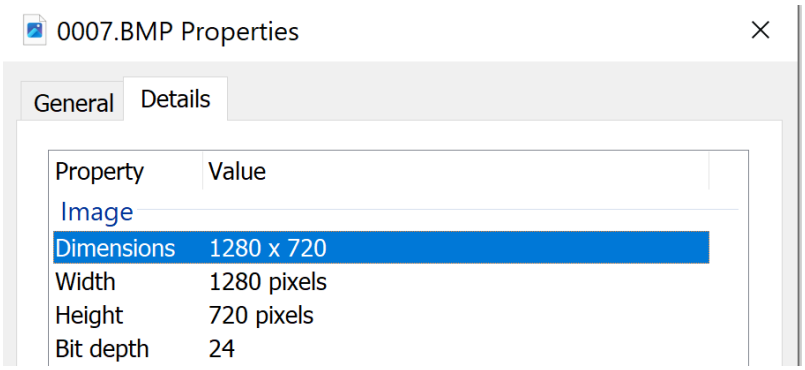


Figure 1.6: The output dimensions

1.1.2 Bootloader Development

Now, after testing the mentioned project, proceed step by step according to *video number 20*, and create the bootloader (FSBL) and Boot.bin file for this project. Place it in the root directory and the first partition of the primary micro SD drive. Set the **boot jumper** on the board to the SD position and power up the board to automatically load the camera template project and image saving into the micro SD. The FPGA will be programmed and the program will run.

1.2 Image Processing Module Development and Simulation

In this part, we dive into designing custom image processing modules in Verilog. You will develop a grayscale conversion module and a Sobel edge detection module, testing both with static images in simulation. This part focuses on crafting and verifying hardware modules before they hit the live system.

You will complete two Verilog modules—**gray.v** and **sobel.v**—using the provided testbenches **tb_gray.v** and **tb_sobel.v (see starter kit)**. In Part 1.2.1, **gray.v** reads each 24-bit RGB pixel from your part 1.1 BMP file and outputs an 8-bit intensity by averaging R, G, and B; the testbench then writes output_gray.bmp in simulation. In Part 1.2.2, **sobel.v** consumes the same BMP input, buffers prior image lines in BRAM, and performs a 3×3 Sobel convolution—computing horizontal and vertical gradients Gx and Gy, summing their absolute values, clamping to 0–255, and replicating across RGB—to produce output_sobel.bmp via **tb_sobel.v**.

The Sobel operator detects edges by convolving a grayscale image $f(x, y)$ with two discrete 3×3 kernels that approximate the image's spatial derivatives. The horizontal kernel Kx and vertical kernel Ky are defined as:

$$G_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \times A \quad , \; G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \times A$$

Convolving yields

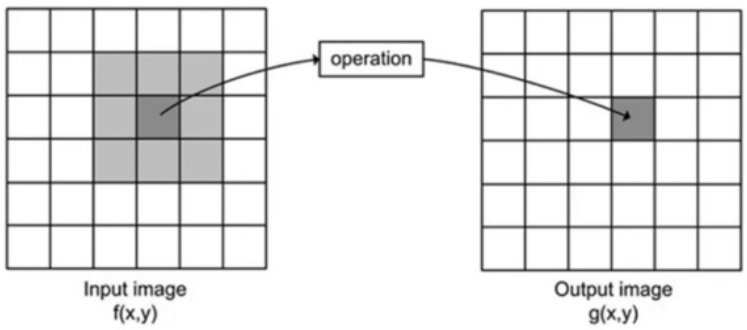


Figure 1.7: Sobel kernel

$$G_x = \sum_{u=-1}^{1} \sum_{v=-1}^{1} K_x(i,j) \cdot P(i+u,\ j+v), \quad G_y = \sum_{u=-1}^{1} \sum_{v=-1}^{1} K_y(i,j) \cdot P(i+u,\ j+v)$$

Rather than computing $G = \sqrt{G_x^2 + G_y^2}$ hardware designs sum absolute values:

$$G = \mid Gx \mid + \mid Gy \mid$$

which yields crisp edges with minimal arithmetic resources.

Because streaming pixels arrive one at a time, you must buffer the immediately needed data. **BRAM line buffers** store the previous rows, while a three-element **shift register** holds the latest three grayscale samples of the current row. On each clock, the newest sample shifts into the register, the line buffers output aligned pixels from previous rows, and these nine samples $\{p_{00}p_{01} \dots p_{22}\}$ feed parallel arithmetic units to compute Gx and Gy in a single cycle. Border pixels (first two rows/columns) are zero-padded by gating out-of-range reads, implementing **zero padding** without extra memory. Finally, pipelining these adders, subtractors, and absolute-value units ensures one-pixel-per-clock throughput once the pipeline is primed.

1.2.1 Grayscale Conversion Module

The provided **gray.v** skeleton declares input and output ports matching **tb_gray.v**,

Your task is to implement, in **gray.v**, a simple average:

$$gray \;=\; \frac{(R+G+B)}{3}$$

using integer adders and a divide-by-3 operation that synthesis will map to LUTs or DSP slices effectively. Verify your design by inspecting simulated waveforms over several pixels and confirming that output_gray.bmp opens as a correct grayscale image.

1.2.2 Sobel Edge Detection Module

The provided **sobel.v** skeleton matches **tb_sobel.v**.

In **sobel.v**, you must:

1. Instantiate or inline your grayscale logic to produce an 8-bit intensity.
2. Implement **BRAMs** line buffers for the prior rows.
3. Create a three-word shift register (delay line) for the current row's latest pixels.
4. On each clock, assemble $\{p_{00} \dots p_{22}\}$ and compute kernel for a pixel:

then take absolute values (e.g via conditional sign-based negation), sum them, clamp the result to 0–255, and replicate across RGB channels.

Simulate **tb_sobel.v** to generate output_sobel.bmp, where clear, high-contrast edges should delineate object boundaries in your original image.

1.3 Hardware Integration and Real-Time Processing

This part brings your modules to life in a real-time setup. You'll integrate the grayscale and Sobel edge detection modules into the hardware, processing live video and displaying the edge-detected output on an HDMI monitor. This part highlights your modules' real-world performance in a hardware-accelerated pipeline.

Please watch below videos before continuing to the procedure:

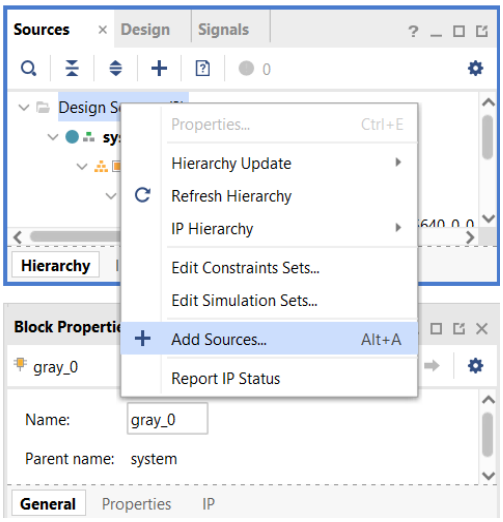
- *3 parts of Video number 15 "AXI DMA"*
- *First part of Video number 18 "micro SD card"*
- *Second and third part of Video number 18 "Image Processing in PL-HDMI out"*
- *Both parts of Video number 12 "DDR3-HP ports-AXI Master MM (PL)"*

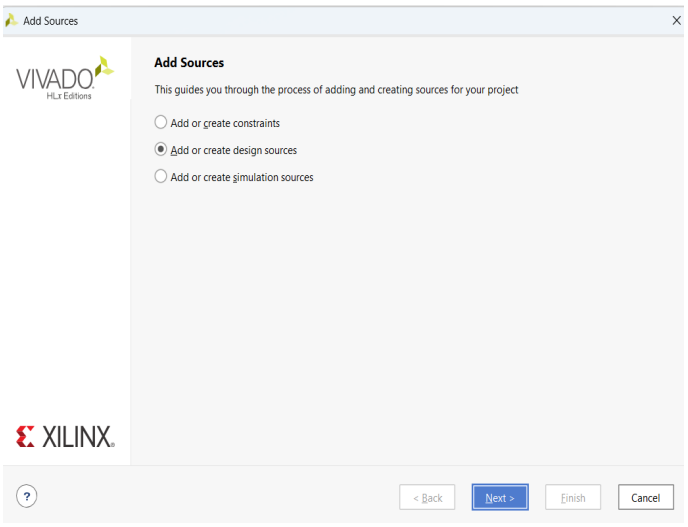
During this part, you''ll add your gray module to the block design to create a gray video result. Then, you'll be adding the Sobel module to the stream to get the final Sobel effect in the output video.

1.3.1 Grayscale Module Integration

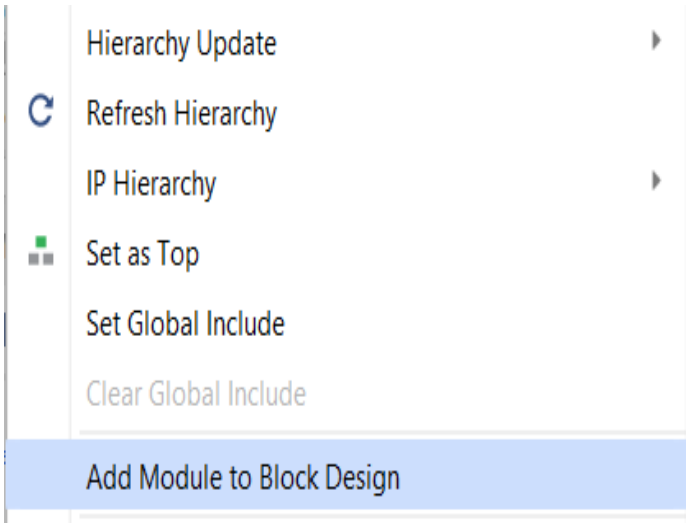
In this section, we will implement the gray.v module that was simulated in part 1.2 of the design diagram in the template project.

So first open *the template project* and then you should add gray.v in design sources:



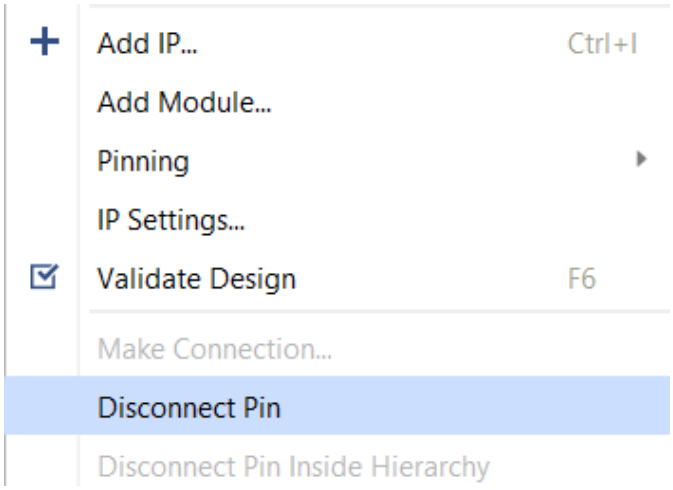


After it, you should add **gray.v** in block design, you can do it by right click on the **gray.v** and choose the **Add module to block design** item:

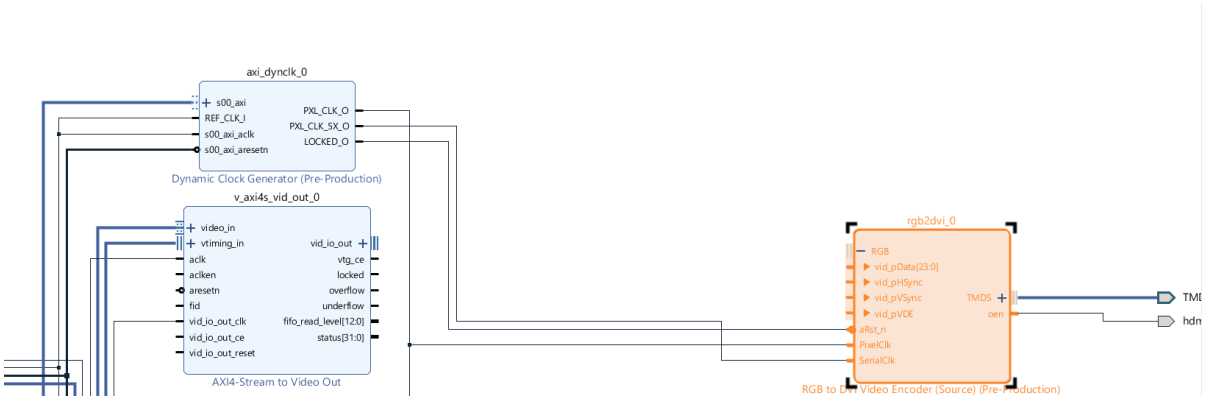


In the block design, you can find the **gray** block and position it between **AXI4-Stream to Video Out** and **RGB to DBI Video Encoder**.

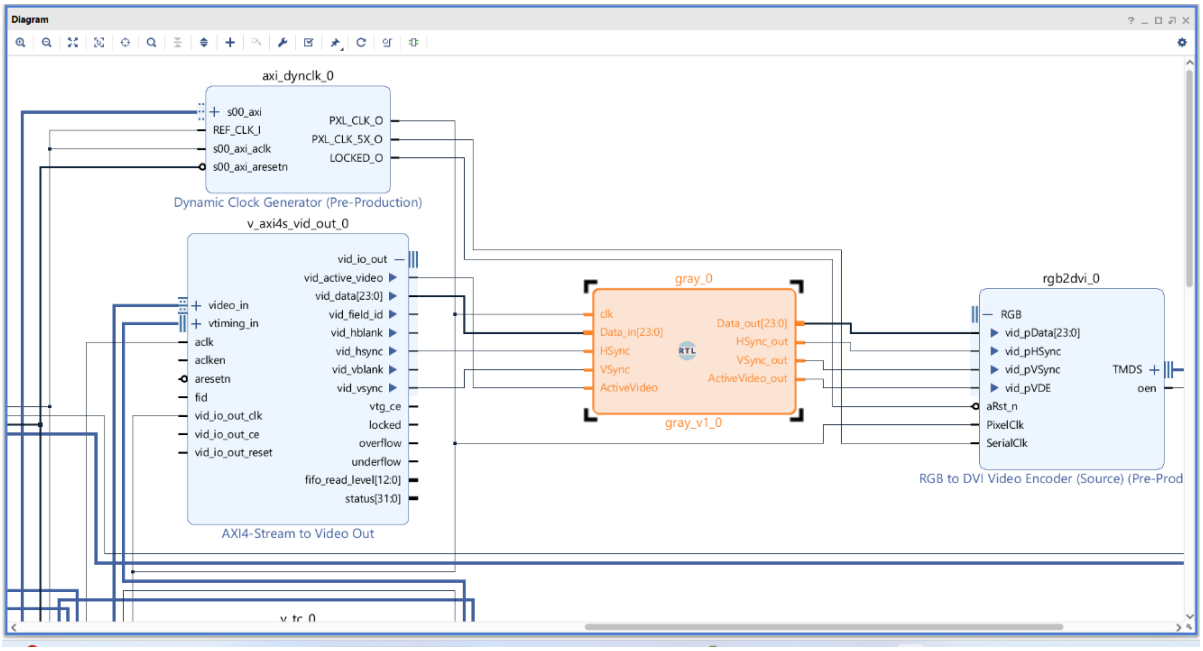
Note: To disconnect **AXI4-Stream to Video Out** from **RGB to DBI Video Encoder,** right click on the pin names and choose **disconnect pin**. It avoids removing extra connections.



This is the expected outcome after disconnecting routes:

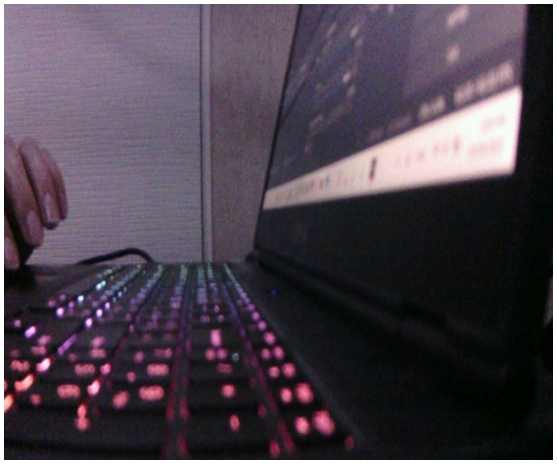


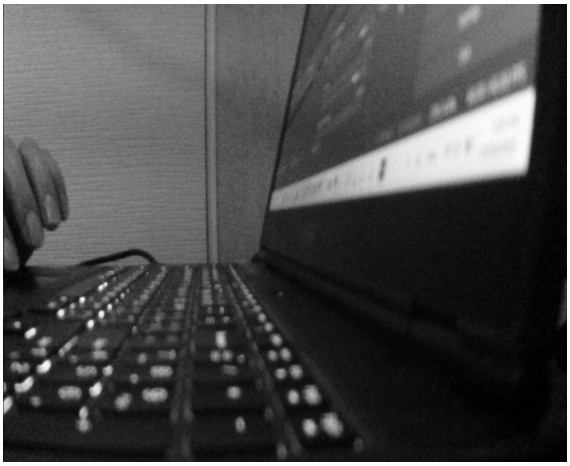
Then put **gray** block between them and route the pin like next picture: **(note:clk -> PXL_CLK_O)**



Performing synthesis, implementation, and generation of a bitstream is necessary before **Exporting hardware** from the File tab. The final step is to launch the SDK. **(See Appendix)**
Then run the project if the gray.v module is correct and you have done all steps right, finally, you should see a **gray live video** on the monitor. Otherwise, check your gray.v and all the last steps.

The final result should be a live gray video:

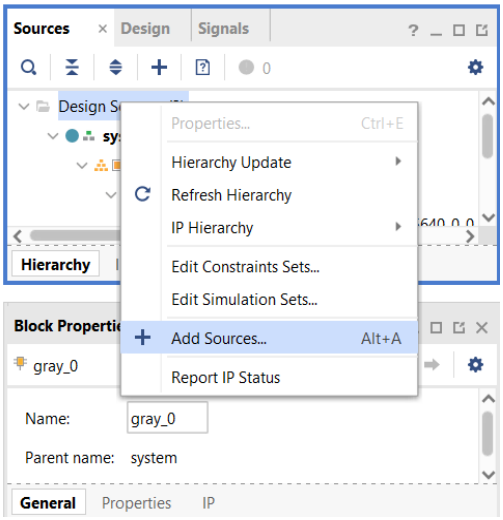


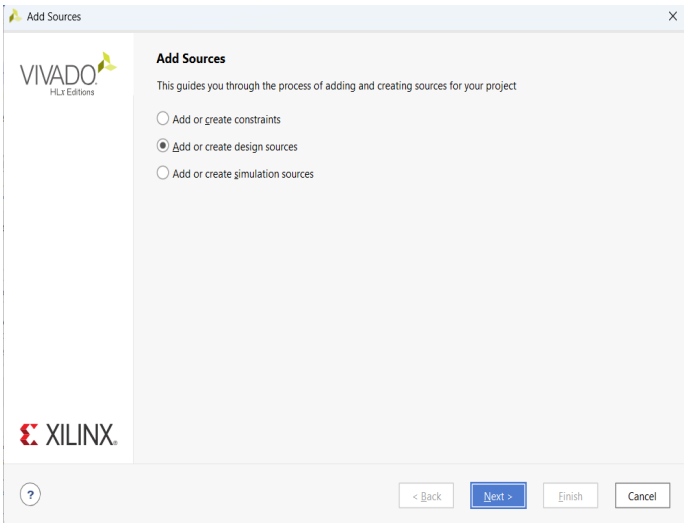


1.3.2 Sobel Module Integration and Testing

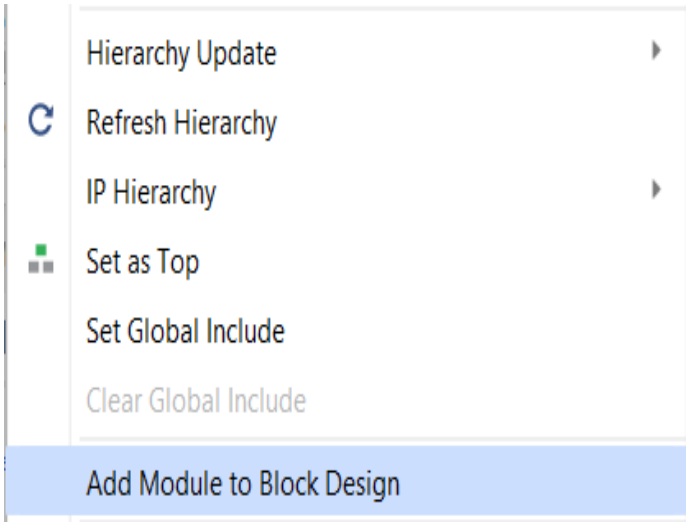
In this part, you will add sobel.v module that was simulated in part 1.2, in the design diagram of the template project.

So, first you should add sobel.v in design sources:



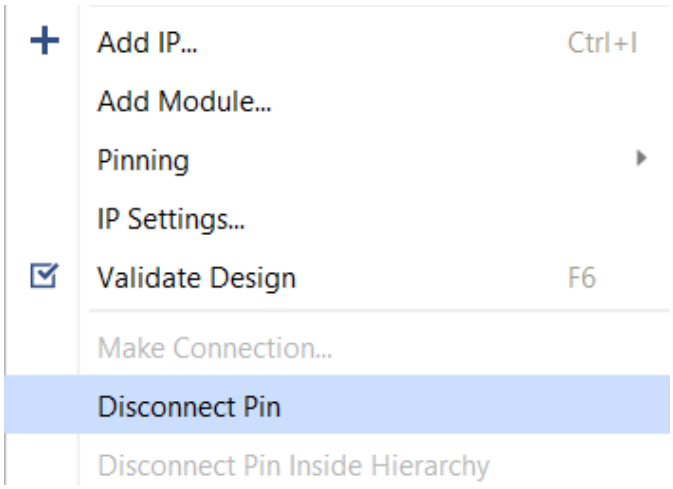


After it, you should add **sobel.v** in block design, you can do it by right click on the **sobel.v** and choose the **Add module to block design** item:

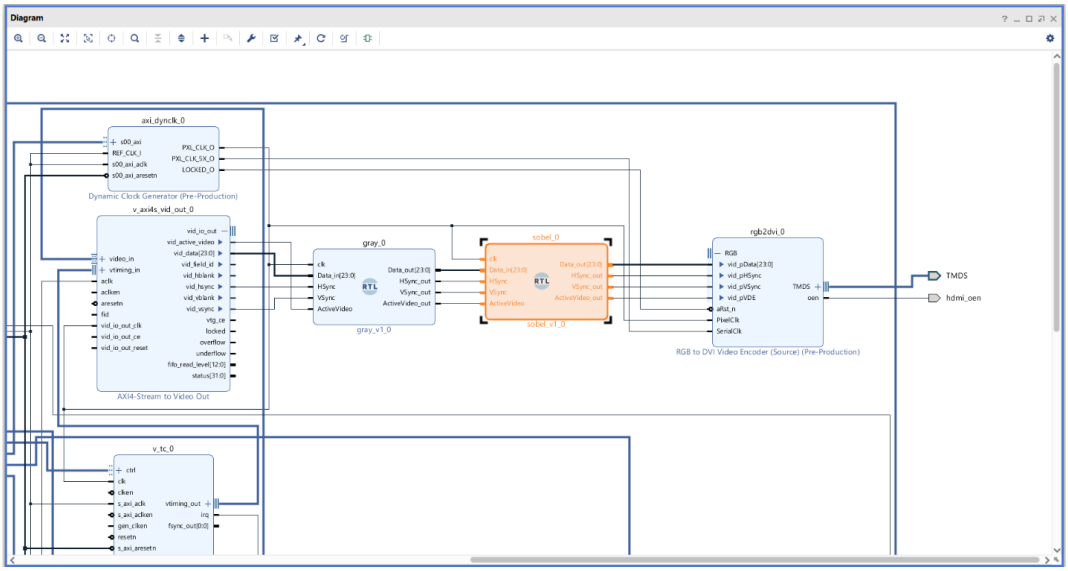


In the block design, you can find the **sobel** block and position it between **gray block** and **RGB to DBI Video Encoder**.

Note: To disconnect **gray block** from **RGB to DBI Video Encoder**, right click on the pin names and choose **disconnect pin**. It avoids removing extra connections.

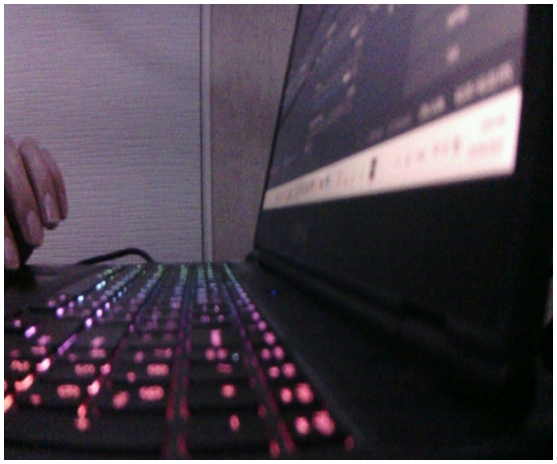


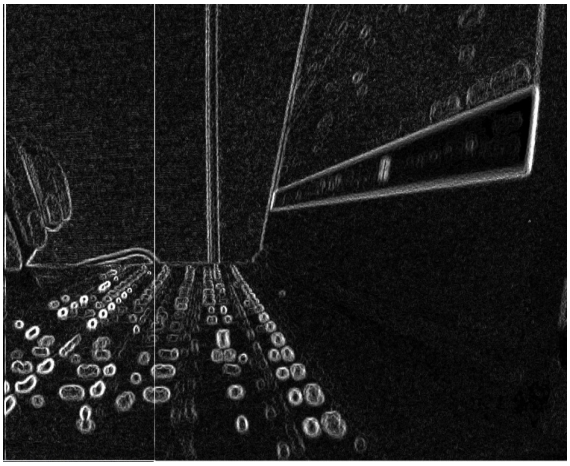
Then put **sobel** block between them and route the pin like next picture:**(note:clk -> PXL_CLK_O)**



Performing synthesis, implementation, and generation of a bitstream is necessary before **Exporting hardware** from the File tab. The final step is to launch the SDK. (See Appendix)
Then run the project if your sobel.v module is correct, and you have done all the steps right, and finally, you should see a Sobel-filtered live video on the monitor. Otherwise, check your sobel.v and all last steps.

The final result should be a live video that has the Sobel filter:





1.3.3 Bonus

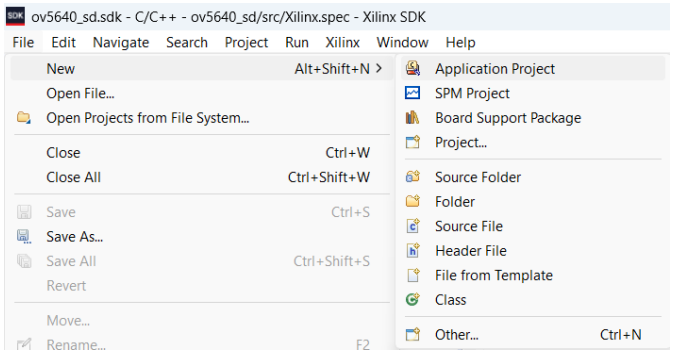
Design an IP core for your Sobel and Gray processing module. After creating the IP core, integrate it into your system block design to visualize the connections and functionality. Once integrated, proceed to extract the video that has been processed through the Sobel filter from this completed design.

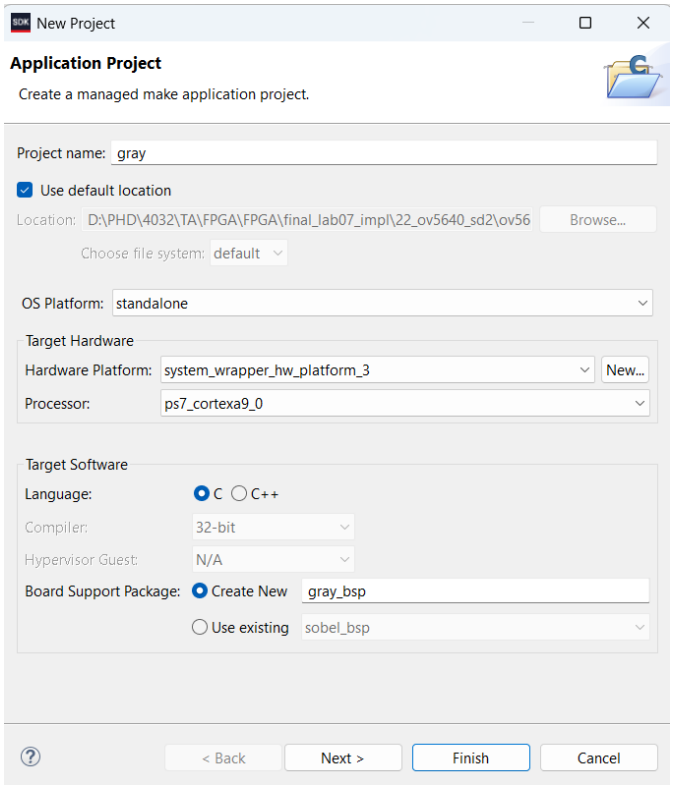
1.3.4 Bonus

Modify the block design and SDK code to save the Sobel filtered image upon pressing the PS KEY1 button, and save it to the SD card.

1.4 Appendix

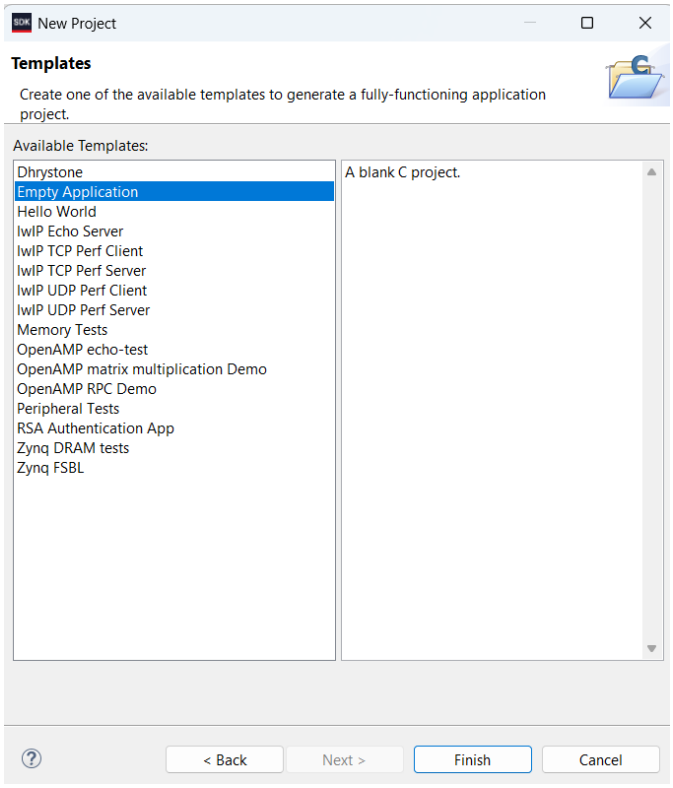
The final step is to launch the SDK. Start by **creating a new application project** and then follow the steps given below.



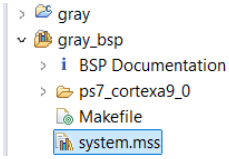


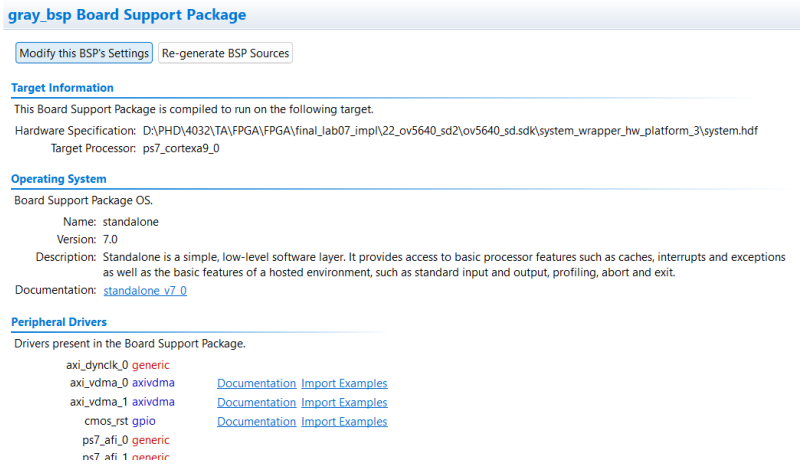
Note: Make sure to select **the last Hardware Platform**. When you change the foundation in PL, a new platform is generated. Therefore, it is important to choose the last Hardware Platform.

In the next step, choose Empty Application and finish it.

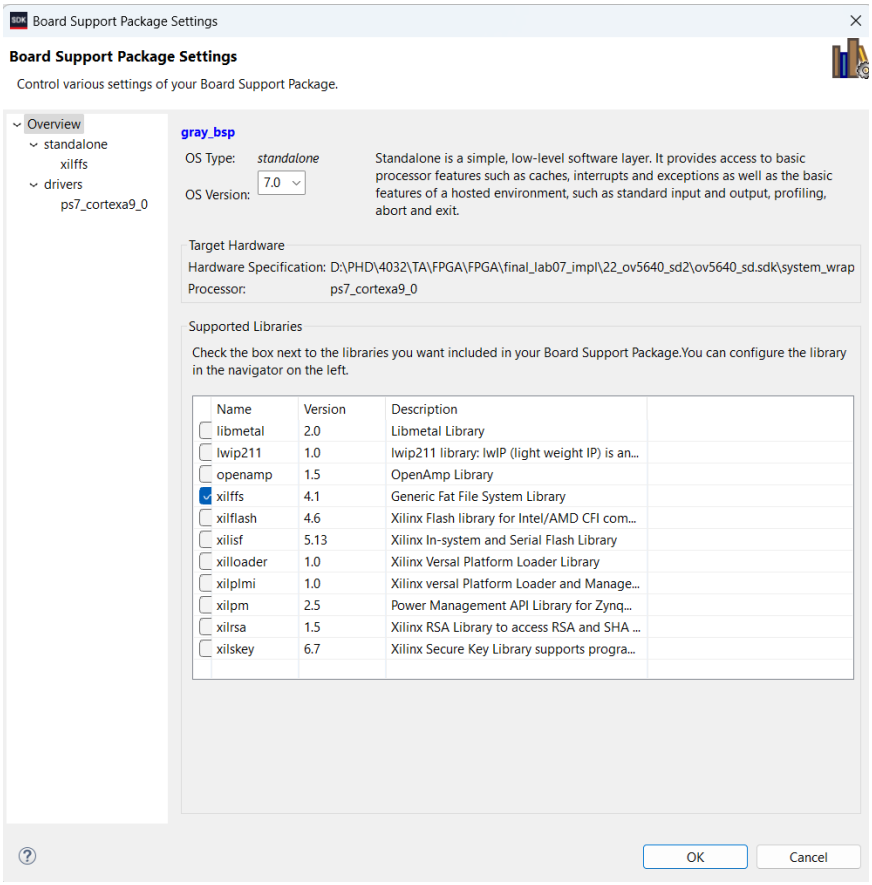


The next step is to configure **gray_bsp** or **sobel_bsp**, which requires the following steps:

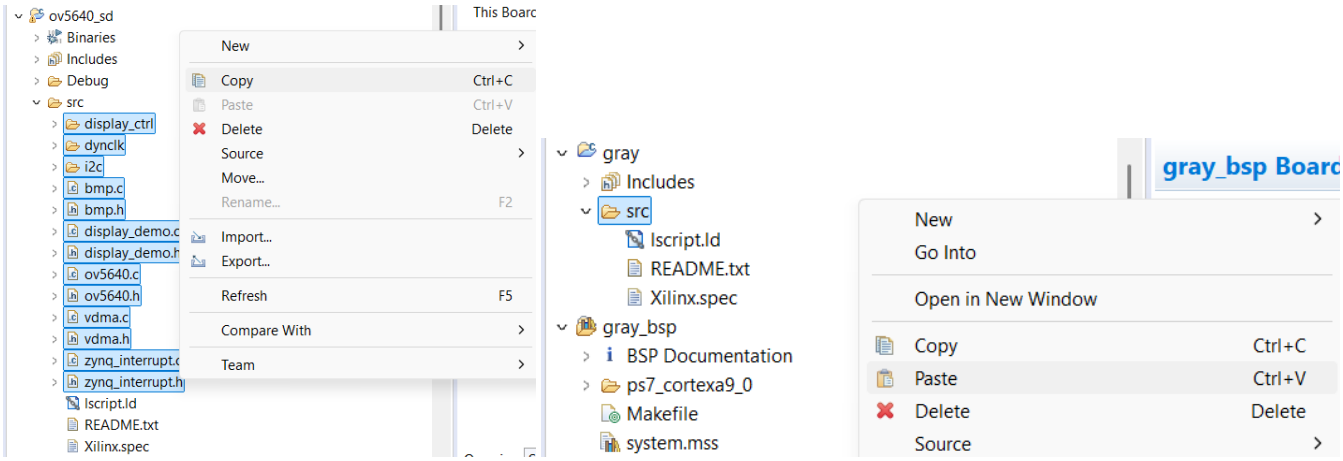




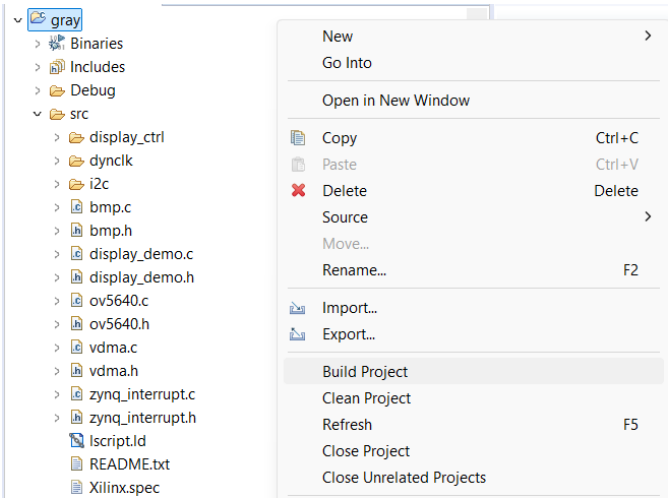
The **xilffs** library should be chosen in this step:



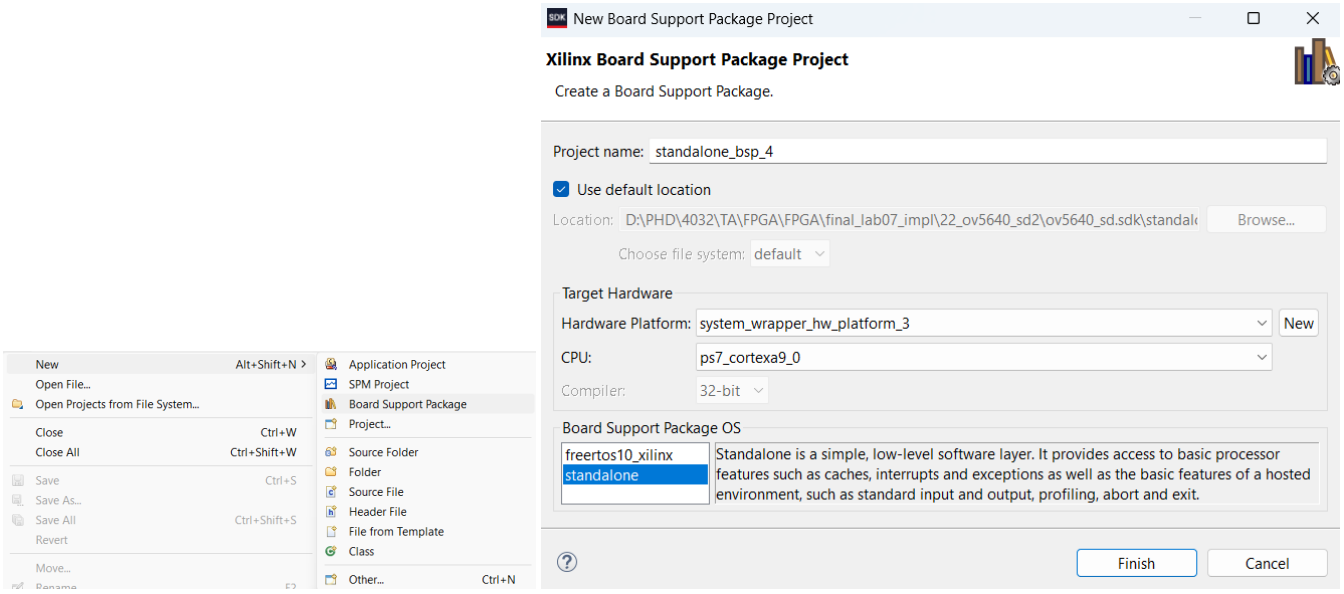
After that, you should **add source files from ov5640_sd to gray or sobel project**, so follow below steps:



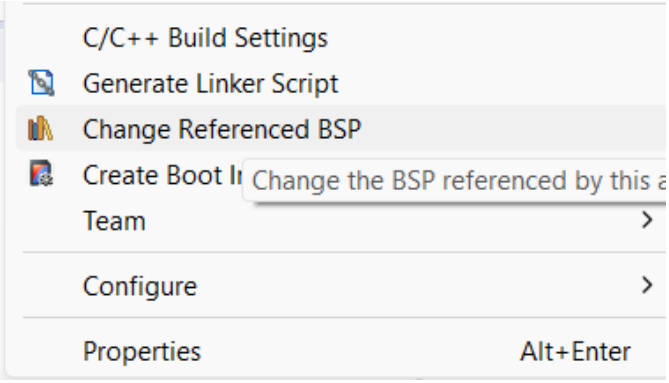
Finally, right click on **gray** or **sobel project** and select build project:



Note: Every time you generate bitstream and export Hardware,I t is very important to make sure that you are using **the last version of the platform** in SDK, so if a new platform was generated, you should **Create a new bsp** with following steps:



Select your new BSP for your project by right-clicking on the project and choosing **Change Referenced BSP**.



Rebuild the project and run it again to test your new version code. Keep doing it until you get results!

Department of Electrical Engineering